Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине Алгоритмы и структуры данных
Тема работы FIFO, LIFO, поиск в ширину и поиск в глубину
Обучающийся Черненко Андрей Дмитриевич
Группа K4111c
Образовательная программа Разработка и эксплуатация промышленного бэкенда

Обучающийся _____ _____ Черненко А. Д.
(дата) (подпись) (Ф.И.О.)

Руководитель _____ _____ Мусаев А. А.
(дата) (подпись) (Ф.И.О.)

Санкт-Петербург, 2026

Содержание

ВВЕДЕНИЕ **2**

1 Задание 1. Проверка правильности скобок **2**

1.1 Условие 2

1.2 Алгоритм решения 2

1.3 Результаты проверки 3

2 Задание 2. Проверка трех типов скобок **4**

2.1 Условие 4

2.2 Алгоритм решения 4

2.3 Результаты проверки 5

3 Задание 3. Граф станций **5**

3.1 Условие 5

3.2 Описание графа 5

3.3 Поиск в ширину 6

3.4 Поиск в глубину 7

4 Сравнение BFS и DFS **8**

ЗАКЛЮЧЕНИЕ **8**

ВВЕДЕНИЕ

Целью лабораторной работы является изучение структур данных LIFO и FIFO, а также алгоритмов поиска в глубину и в ширину.

В работе решаются следующие задачи:

1. проверка правильности расстановки скобок и поиск позиции первой ошибки;
2. расширение проверки на круглые, квадратные и фигурные скобки;
3. построение графа станций и поиск пути между вершинами с помощью DFS и BFS;
4. сравнение алгоритмов по скорости, памяти и назначению.

Все программы реализованы на языке C#.

1 Задание 1. Проверка правильности скобок

1.1 Условие

Необходимо определить, является ли введенная скобочная структура правильной. Для правильной последовательности каждая открывающая скобка должна иметь соответствующую закрывающую, а порядок закрытия должен быть обратным порядку открытия. Дополнительно требуется определить порядковый номер первого символа, нарушающего правильность.

1.2 Алгоритм решения

Для решения используется стек LIFO. При чтении открывающей скобки она помещается в стек. При чтении закрывающей скобки проверяется верхний элемент стека:

1. если стек пуст, текущая закрывающая скобка является ошибочной;
2. если верхний элемент не соответствует текущей скобке, ошибка находится в текущей позиции;

3. если скобки соответствуют друг другу, верхний элемент удаляется.

После обработки всей строки пустой стек означает правильную последовательность. Если в стеке остались открывающие скобки, первой ошибкой считается позиция самой ранней незакрытой скобки.

```
static (bool correct, int errorPosition) CheckRoundBrackets(
    string text)
{
    Stack<int> stack = new();

    for (int i = 0; i < text.Length; i++)
    {
        if (text[i] == '(')
            stack.Push(i + 1);
        else if (text[i] == ')')
        {
            if (stack.Count == 0)
                return (false, i + 1);

            stack.Pop();
        }
    }

    if (stack.Count > 0)
        return (false, stack.Min());

    return (true, -1);
}
```

Листинг 1: Проверка круглых скобок

1.3 Результаты проверки

Таблица 1: Проверка последовательностей круглых скобок

Последовательность	Результат	Позиция ошибки
( )	правильная	---
(( )) ( )	правильная	---
) (	неправильная	1
( )) (	неправильная	3
((( )	неправильная	1

Алгоритм выполняет один проход по строке. Его временная сложность составляет $O(n)$, а сложность по памяти --- $O(n)$ в худшем случае, если все символы являются открывающими скобками.

2 Задание 2. Проверка трех типов скобок

2.1 Условие

Необходимо расширить программу для работы с круглыми, квадратными и фигурными скобками: `()`, `[]` и `{}`. Закрывающая скобка должна соответствовать последней открывающей скобке в стеке.

2.2 Алгоритм решения

В стек помещается пара: тип открывающей скобки и ее позиция. Для каждой закрывающей скобки вызывается функция `Matches`, которая проверяет, образует ли она пару с верхним элементом стека.

```
static bool Matches(char open, char close)
{
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

static (bool correct, int errorPosition) CheckBrackets(
    string text)
{
    Stack<(char bracket, int position)> stack = new();

    for (int i = 0; i < text.Length; i++)
    {
        char symbol = text[i];

        if (symbol is '(' or '[' or '{')
            stack.Push((symbol, i + 1));
        else if (symbol is ')' or ']' or '}')
        {
            if (stack.Count == 0)
                return (false, i + 1);

            var top = stack.Pop();
            if (!Matches(top.bracket, symbol))
                return (false, i + 1);
```

```
        }
    }

    return stack.Count == 0
        ? (true, -1)
        : (false, stack.Peek().position);
}
```

Листинг 2: Проверка трех типов скобок

2.3 Результаты проверки

Таблица 2: Проверка последовательностей разных типов скобок

Последовательность	Результат	Позиция ошибки
`([])\{\}`	правильная	---
`([])`	правильная	---
`([)]`	неправильная	3
`\{[(])\}`	неправильная	4
`((]`	неправильная	3

Использование стека гарантирует, что вложенные скобки проверяются в правильном порядке. Например, последовательность `([)]` ошибочна, потому что после открывающей круглой скобки последней была открыта квадратная скобка, но встретилась круглая закрывающая.

3 Задание 3. Граф станций

3.1 Условие

Для условной карты метро построить граф и реализовать поиск пути между двумя станциями в ширину и в глубину. Вершины графа соответствуют станциям, ребра --- переходам между соседними станциями.

3.2 Описание графа

Рассмотрен следующий неориентированный граф:

Таблица 3: Список смежности графа станций

Станция	Соседние станции
A	B, C
B	A, D
C	A, D, E
D	B, C, F
E	C, F
F	D, E

Требуется найти путь от станции A до станции E. Порядок соседей в списке фиксирован, чтобы результат обхода был воспроизводимым.

3.3 Поиск в ширину

Поиск в ширину BFS использует очередь FIFO. Сначала посещаются вершины, находящиеся на расстоянии одного ребра от начальной, затем на расстоянии двух ребер и так далее. Для невзвешенного графа BFS находит кратчайший путь по количеству ребер.

```
static List<string> Bfs(
    Dictionary<string, List<string>> graph,
    string start, string target)
{
    Queue<string> queue = new();
    Dictionary<string, string?> parent = new();
    HashSet<string> visited = new();

    queue.Enqueue(start);
    visited.Add(start);
    parent[start] = null;

    while (queue.Count > 0)
    {
        string current = queue.Dequeue();

        if (current == target)
            break;

        foreach (string next in graph[current])
        {
            if (visited.Contains(next))
                continue;

```

```
            visited.Add(next);
            parent[next] = current;
            queue.Enqueue(next);
        }
    }

    return RestorePath(parent, start, target);
}
```

Листинг 3: Поиск пути в ширину

Для заданного графа результат BFS:

$$A \to C \to E.$$

Длина пути равна двум ребрам.

3.4 Поиск в глубину

Поиск в глубину DFS использует стек LIFO или рекурсию. Алгоритм продолжает идти по выбранной ветви максимально глубоко и только затем возвращается к ранее посещенным вершинам.

```
static bool Dfs(
    Dictionary<string, List<string>> graph,
    string current, string target,
    HashSet<string> visited, List<string> path)
{
    visited.Add(current);
    path.Add(current);

    if (current == target)
        return true;

    foreach (string next in graph[current])
    {
        if (!visited.Contains(next) &&
            Dfs(graph, next, target, visited, path))
            return true;
    }

    path.RemoveAt(path.Count - 1);
    return false;
}
```

Листинг 4: Поиск пути в глубину

При указанном порядке соседей DFS сначала идет по ветви $A \to B \to D \to F \to E$. Найденный путь содержит четыре ребра. Он существует, но не является кратчайшим.

4 Сравнение BFS и DFS

Таблица 4: Сравнение поиска в ширину и поиска в глубину

Критерий	BFS	DFS
Структура	Очередь FIFO	Стек LIFO или рекурсия
Время	$O(V + E)$	$O(V + E)$
Память	$O(V)$	$O(V)$
Кратчайший путь	Находит в невзвешенном графе	Не гарантирует
Основное применение	Кратчайшие маршруты, уровни	Обход, компоненты, циклы

Здесь V --- число вершин графа, E --- число ребер. Оба алгоритма имеют одинаковую асимптотическую сложность, однако используют разные стратегии. BFS обычно требует больше памяти на графах с широкими уровнями, а DFS может расходовать значительную глубину стека на длинных цепочках.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализована проверка правильности скобочных последовательностей с использованием стека LIFO. Программа определяет соответствие круглых, квадратных и фигурных скобок и выводит позицию первого нарушения.

Также построен граф станций и реализованы алгоритмы поиска в ширину и в глубину. BFS нашел кратчайший путь от A до E: $A \to C \to E$. DFS прошел по первой доступной ветви и построил более длинный путь: $A \to B \to D \to F \to E$.

Таким образом, BFS целесообразно выбирать, когда требуется кратчайший маршрут в невзвешенном графе, а DFS --- когда важен полный обход

графа, поиск циклов или обработка компонент связности.